

Windborne Systems - Firmware Wizard Submission

Here are my answers to the question challenges :)

1. I have so many cool things, it's hard to decide! Since I mostly talk about my FSAE team experiences, I'll mention one from back in highschool. In my senior year I helped create the curriculum for a new Electronics Workshop course, along with the teachers. The final project was a dual-motor drive robot with a color sensor, ToF distance sensor, and Hall-effect sensor which was meant to drive across a grid to detect colored squares and hidden magnets, controlled by an RPI Pico. The brushed DC motors we chose to drive our robots were frustratingly inconsistent, with very non-linear rpm and inconsistent manufacturing making it impossible to drive in a straight line. To combat this, I used a 9-axis IMU to perform drive correction by measuring the angular velocity along the vertical axis (yaw rate), applying a correction-factor to the motor power. I then implemented a PID loop to prevent oscillations and over-corrections in turning, allowing my bot to traverse much faster and more reliably :) (I also used the IMU to assist with 90-degree turning by integrating the yaw rate over time, but I found that the drift from integration resulted in much less precise turning after 4-5 turns, so I reverted to just using the distance sensor to find when the bot was pointing straight against the wall and stop the motors, albeit a slower and clunkier solution).

Here's a link to a repository with the code and some pictures: <https://github.com/bvngee/SAR>

2. Ooooh don't get me started! I'm a *passionate* Neovim user. I'm known as the Vim guy on my FSAE team, and I will take every opportunity I get to dunk on IDE users :) I have good reasoning I promise, it's justified! Here's what I'd say:
 1. I love nothing more than feeling like I own and understand every bit of the codebase I'm working on, and my Neovim setup gives me just that. Want to go somewhere? There's a hotkey for that. Not sure where you are? `␣sf` to search for files, or `␣fb` for an interactive file browser. Want to grep for something? There's a UI for that. Want to view a list of LSP diagnostics? There's a UI for that. Want to find a list of references to a symbol? There's a UI for that. It lets me feel like I *own* the code that I'm working on, whereas IDEs I used in the past made me scared of finding out where the header files are coming from or what magic it's doing to set the environment variables for me.
 2. I love open and compatible build systems, and using Neovim enhances that experience. I get fantastic C/C++ editor support thanks to the `compile_commands.json` specification, which is implemented by the language server I choose to use. But hey, if there's something I don't like about it, I can always switch from CMake to Meson, or choose another LSP! The setup works for any build system, language, compiler version, etc that implements the protocols Neovim expects.
 3. I love open source software, and Neovim is such a role model for this. You can just tell it's made by a group of nerds just like you that want to use and write good software. In the issue trackers, the plugin ecosystem, the new features, and the minimalism.

Damn I sound like I'm selling Neovim rn. Anyways, [here's my configuration](#), as part of my NixOS config repo. You can browse the commit history if you're interested :) (this is really my third neovim config iteration, but hey who's counting)

3. (Note: I realize I wrote my answer to this question assuming you meant that a microcontroller on the PCB was resetting every few hours, not that a circuit on the PCB was resetting. I'll leave my answer and elaborate a bit after.) My first instinct is a memory leak, stack space overflow, or heap

memory exhaustion. Unfortunately you can't just Valgrind embedded code (as far as I know at least), unless some unit tests exist that can be run on a host OS. But I'll assume not for now.

- I would expect that the HAL/RTOS/SDK being used has a fault handler that leads a memory error into a halt or busy loop. Is the code restarting immediately, or stopping? Could it be some serious power supply noise that drops the voltage low enough to shut off the MCU, or just a full on temporary power loss?
- Assuming it stops and doesn't restart, I'd check the HAL/RTOS for a memory usage API, and insert some printf's of usage statistics in commonly run code paths. See if numbers go up.
- Check the git log. How long has this been happening for? Do we know? What features were added recently or major changes made that could've caused this? Were there any changes to the environment that the device is running in?
 - As an example: grep the codebase for malloc/calloc/new, see if there's an obvious location that could be suspiciously run conditionally, or that could accumulate memory if used incorrectly (a C++ destructor not getting called, for example)

Of course that assumes a firmware-only bug. Are there any changes to the physical PCB when the reset happens? Anything hot, or clicking?

4. I don't think this is necessarily the fault of any firmware engineers specifically, because it's good engineering to go with a well-supported architecture, but I think we default to RTOSes (or in general, task-switching event loops, like kernel threads) too much. I wish there were more embedded solutions with coroutine-based event loops, with coroutine yielding as the fundamental unit of concurrency for asynchronous I/O. Meaning, instead of making a thread that does a blocking SPI write and the "kernel" (RTOS) switches off of that thread manually (and possibly suboptimally), a coroutine would initiate a non-blocking SPI write, yield to the event loop, and register an interrupt with the event loop that would indicate when to resume back into the coroutine. (Modern languages implement this by fancy compiler features that turn your functions into state machines at each yield/resume point, but I think there's other ways as well). Perhaps there's something I don't understand about why this doesn't work as well in an embedded context, but I do wonder if RTOSes are more overkill than we think!
5. Absolutely :) The FSAE environment (Formula Student, Electric) is very conducive to last-second rushed fixes that are necessary for your racecar to pass inspection and compete. This past year was the first year *ever* that my team, Formula Slug, fully passed inspections and drove our car at competition, and it was partially thanks to a few hacky fixes I made to our throttle control firmware. During the "EV Active" inspection, they check to make sure that your redundant Accelerator Pedal position sensors are actually redundant; i.e. your code will disable the motor if there's a discrepancy between the two. Ours didn't. Then, they make sure your code will re-enable the motor once the sensors are back in range again. Again, ours didn't. I ran to grab my laptop from the paddock, and opened up the code as quickly as I could. Thanks to a bit of help from the electrical captain at the time dealing with weird variable names that I wasn't familiar with yet, I was able to quickly determine that... there was no real *clean* way to add this behavior in that would be maintainable or understandable to others. But the timer was ticking in the most literal sense imaginable, so I chucked in like 6 duplicate if-statements across the codebase to hack together the redundancy checks, and after one more EV active inspection we passed and were able to drive :) ([You can see a video of our car competing here!](#)) I could not be prouder of those quick fixes.